

Del Software Procedural a los Agentes Autónomos: Evaluación Práctica de una Arquitectura Híbrida Basada en LLM para Producción Real

Abstract

Los modelos de lenguaje de gran tamaño (LLM) están introduciendo un nuevo paradigma en el desarrollo de software, desplazando parcialmente la lógica procedural tradicional hacia sistemas guiados mediante lenguaje natural, contexto y herramientas externas. Este trabajo presenta el diseño y evaluación práctica de una arquitectura híbrida de agente autónomo orientada a entornos reales de producción. Durante el desarrollo se evaluaron distintos entornos de inferencia local y cloud, incluyendo OpenAI API, Ollama y vLLM, analizando problemas reales relacionados con latencia, concurrencia, gestión de contexto y viabilidad operativa.

1. Introducción y estado del arte

La automatización tradicional de procesos empresariales se ha basado históricamente en programación procedural y lógica explícita, donde cada nueva funcionalidad requiere implementar métodos, condiciones y flujos específicos. La aparición reciente de modelos de lenguaje de gran tamaño ha modificado parcialmente este paradigma, permitiendo definir objetivos mediante lenguaje natural y delegar parte del razonamiento operativo al modelo.

Durante 2024 y 2025 han aparecido múltiples arquitecturas de agentes autónomos capaces de combinar razonamiento iterativo, uso dinámico de herramientas externas y recuperación contextual. Sin embargo, gran parte de estos trabajos se centran en capacidades conceptuales del modelo y menos en problemas reales asociados a producción.

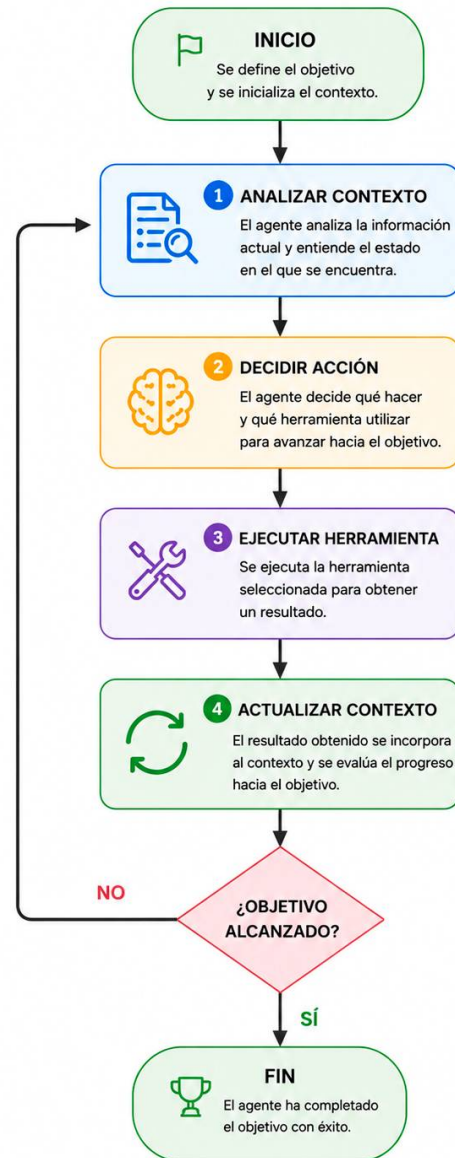
2. Desarrollo

2.1 Arquitectura conceptual del agente

Durante el desarrollo del sistema se observó que el comportamiento esencial del agente podía simplificarse conceptualmente mucho más de lo inicialmente esperado. La principal conclusión obtenida fue comprobar que el núcleo operativo del sistema podía representarse mediante un ciclo iterativo de decisión y ejecución.

Figure 1. Ciclo conceptual de decisión y ejecución del agente autónomo.

```
Inicializar contexto
Definir objetivo
do
  Decisión = LLM(Contexto)
  Resultado = EjecutarHerramienta(Decisión)
  Contexto = ActualizarContexto(Resultado)
while Objetivo no completado
```



El agente analiza el contexto disponible, decide la siguiente acción y ejecuta herramientas externas hasta completar el objetivo solicitado. Este enfoque desacopla el razonamiento del conocimiento específico de negocio y permite reutilizar el mismo núcleo para distintos escenarios mediante prompts y herramientas especializadas.

2.2 Cambio de abstracción

En los sistemas procedurales clásicos, cada nueva funcionalidad requiere implementar nuevos métodos y estructuras de control específicas. En cambio, el agente desarrollado utiliza un núcleo genérico capaz de resolver múltiples tareas mediante contexto, herramientas y lenguaje natural.

Como consecuencia, parte del comportamiento del sistema emerge dinámicamente a partir del razonamiento probabilístico del modelo y no únicamente de lógica rígida programada explícitamente. Este cambio introduce un nuevo nivel de abstracción donde el desarrollador pasa de definir el procedimiento exacto a describir principalmente el objetivo que debe alcanzarse.

2.3 Especialización funcional y herramientas

Inicialmente se planteó utilizar un único modelo avanzado para todas las tareas del sistema. Sin embargo, las limitaciones actuales relacionadas con latencia, coste computacional y concurrencia hicieron inviable esta aproximación para producción real.

La experiencia obtenida permitió concluir que no todas las tareas requieren el mismo nivel de inteligencia ni la misma capacidad de respuesta temporal. Como consecuencia, se adoptó una arquitectura híbrida basada en especialización funcional, utilizando modelos avanzados para razonamiento y herramientas deterministas o modelos ligeros para tareas auxiliares.

2.4 Gestión del contexto

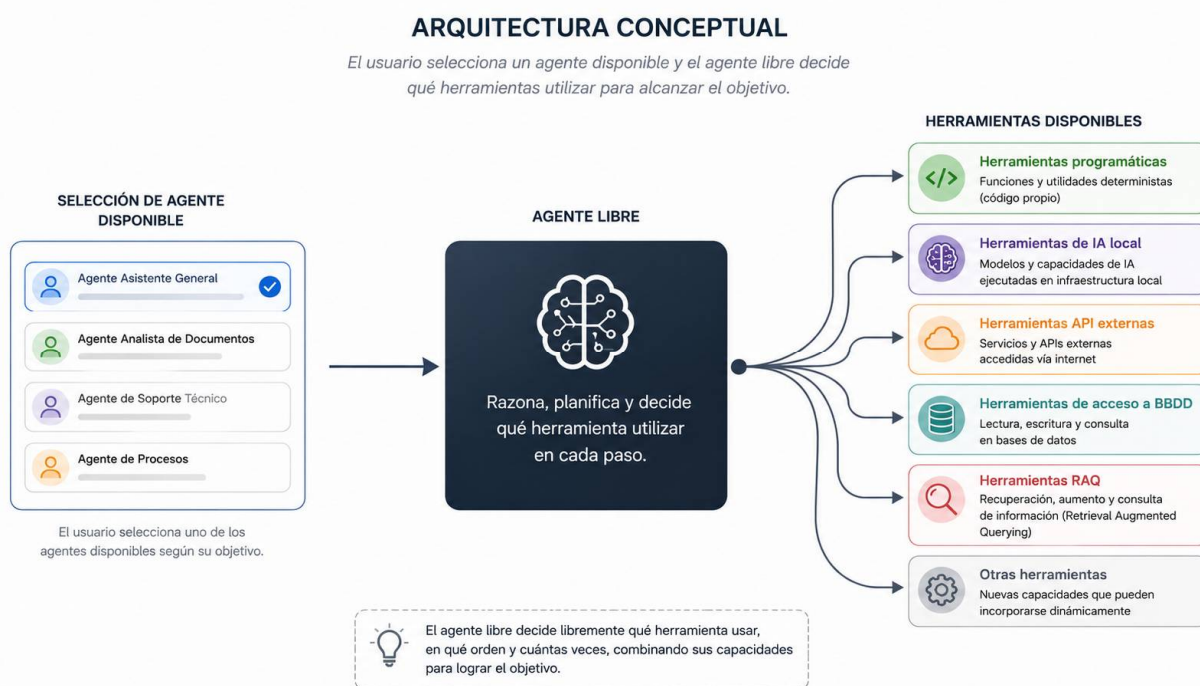
Uno de los principales problemas encontrados durante el desarrollo fue la gestión del contexto en entornos API. El trabajo mediante API obliga a reconstruir manualmente el estado operativo completo del agente en cada iteración, introduciendo limitaciones relacionadas con el número máximo de tokens, el incremento progresivo del coste computacional y la pérdida parcial de contexto.

La experiencia obtenida permitió comprobar que uno de los principales retos actuales de los agentes autónomos no consiste únicamente en generar respuestas correctas, sino en mantener un contexto operativo estable y eficiente durante múltiples iteraciones.

2.5 Inferencia local y despliegue en producción

El sistema fue evaluado utilizando distintos entornos de inferencia. Inicialmente se trabajó mediante OpenAI API debido a su elevada velocidad de respuesta y facilidad de integración. Posteriormente se realizaron pruebas locales utilizando Ollama y modelos Qwen 2.5 con el objetivo de reducir dependencia externa y aumentar privacidad y control operativo.

Las primeras pruebas locales mostraron importantes limitaciones de concurrencia y throughput incluso utilizando hardware de altas prestaciones. Posteriormente se implementó una infraestructura basada en vLLM, CUDA, Docker y WSL, obteniendo mejoras significativas en paralelismo y capacidad concurrente.



3. Resultados

Las pruebas realizadas permitieron comprobar que el sistema es capaz de resolver tareas complejas utilizando un núcleo genérico de decisión basado en lenguaje natural y herramientas externas. En escenarios estándar de procesamiento documental, el agente fue capaz de interpretar documentos, extraer información relevante y generar resultados estructurados de forma consistente.

Sin embargo, durante las pruebas también aparecieron limitaciones importantes relacionadas con pérdidas parciales de contexto, incremento de latencia y degradación de coherencia operativa en ejecuciones largas o altamente concurrentes.

4. Conclusiones

El trabajo realizado demuestra que los agentes autónomos basados en LLM pueden integrarse de forma viable en entornos empresariales reales mediante arquitecturas híbridas orientadas a producción. La principal aportación del proyecto consiste en simplificar conceptualmente el funcionamiento de un agente autónomo a un ciclo iterativo de decisión y ejecución guiado mediante lenguaje natural.

Asimismo, la experiencia obtenida permitió comprobar que los principales retos actuales no residen únicamente en la capacidad del modelo, sino en la gestión eficiente del contexto, la concurrencia, la latencia y el coste computacional necesarios para producción real.

5. Bibliografía

[1] ReAct: Synergizing Reasoning and Acting in Language Models (2022) - Uno de los papers fundacionales en arquitecturas de agentes basados en LLM. Introduce la idea de combinar razonamiento y ejecución de acciones dentro de un mismo ciclo iterativo, concepto muy relacionado con el núcleo do-while utilizado en este proyecto.

<https://arxiv.org/abs/2210.03629>

[2]. A Survey on Large Language Model based Autonomous Agents (2023) - Paper de referencia que realiza una revisión general sobre agentes autónomos basados en LLM, incluyendo arquitectura, memoria, planificación, herramientas y aplicaciones prácticas.

https://www.researchgate.net/publication/373297758_A_Survey_on_Large_Language_Model_based_Autonomous_Agents

[3]. AutoAgents: A Framework for Automatic Agent Generation (2024) - Trabajo centrado en generación automática de agentes mediante LLM y uso dinámico de herramientas externas. Resulta especialmente relevante para arquitecturas reutilizables y workflows autónomos.

<https://www.ijcai.org/proceedings/2024/0003.pdf>

[4]. LLM-Based Agents for Tool Learning: A Survey (2025) - Survey centrado específicamente en agentes capaces de aprender y utilizar herramientas externas. Muy relacionado con el enfoque híbrido utilizado en este trabajo.

<https://link.springer.com/article/10.1007/s41019-025-00296-9>

[5.] Qwen2.5 Technical Report (2024) - Paper técnico oficial de la familia Qwen2.5, modelos utilizados durante las pruebas locales realizadas en este proyecto, especialmente las variantes 7B y 14B. El documento describe arquitectura, entrenamiento, capacidades y optimizaciones de inferencia.

<https://arxiv.org/abs/2412.15115>

[6]. Qwen2.5-1M Technical Report (2025) - Trabajo centrado en la ampliación de contexto de Qwen2.5 hasta un millón de tokens. Resulta especialmente relevante para los problemas de memoria y contexto analizados durante el desarrollo del agente.

<https://arxiv.org/abs/2501.15383>